



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Sistemas de Computación

Trabajo Práctico 3

Modo Protegido

Integrantes

Gonzalez Leahy María Clara

Nuñez Anna

Vigazzi Ignacio Joaquin

UEFI y Coreboot

1) ¿Qué es UEFI? ¿Cómo puedo usarlo? Mencionar una función a la que podría llamar usando esa dinámica

UEFI o Unified Extensible Firmware Interface es una especificación para la arquitectura de firmware que utiliza la computadora para inicializar el sistema y el hardware. Surge como reemplazo del BIOS, aportando mejoras significativas como una interfaz estándar entre el sistema operativo y el firmware, opciones de seguridad avanzadas y la utilización de tablas de particiones GPT en discos en lugar de MBR, permitiendo más particiones de mayor tamaño.

Para utilizar UEFI se pueden crear aplicaciones que corran antes que se cargue el sistema operativo, por ejemplo mediante el uso de Tianocore EFI Development Kit II, inicialmente desarrollado por Intel.

Además, se pueden aprovechar:

- Servicios de Arranque: Disponibles en el pre arranque del sistema, permiten entre otras cosas cargar imágenes ejecutables mediante las funciones LoadImage() y StartImage().
- Servicios de Tiempo de Ejecución: Permanecen disponibles después de que se cargue el sistema operativo, permitiendo comunicación con el firmware por ejemplo para acceder al reloj en tiempo real o leer y escribir variables persistentes del firmware como configuraciones de arranque mediante GetVariable() y SetVariable().

2) Mencionar casos de bugs de UEFI que puedan ser explotados

Hybrid Petya (2025): Variante de un ransomware del mismo nombre, al que se le incluyó un componente que explotaba la vulnerabilidad CVE-2024-7344, permitiendo evadir el protocolo de Secure Boot y ejecutar código de arranque que cifra la Master File Table del sistema de archivos ntfs de la instalación de Windows del usuario

Logo Fail (2023): Mediante vulnerabilidades en el DXE o Driver Execution Environment se podía reemplazar las imágenes utilizadas para mostrar los logos del sistema con código malicioso que sería ejecutado al momento de arranque Bypass de Secure Boot mediante NVRAM: Explotando la vulnerabilidad CVE-2025-3052 se puede manipular una variable de NVRAM o memoria no volátil para ejecutar código no firmado con privilegios administrativos antes de que cargue el sistema operativo.

3) ¿Qué es Converged Security and Management Engine (CSME) , the Intel Management Engine BIOS Extension (Intel MEBx)?

CSME es un subsistema de los procesadores Intel modernos. Implementado como un microcontrolador independiente del procesador principal, tiene acceso a memoria, almacenamiento y acceso a red incluso cuando el sistema está apagado siempre que el dispositivo tenga alimentación.

Su principal uso es para la implementación de funciones de seguridad como Secure Boot o cifrado de datos. Además, se utiliza para gestión remota en entornos corporativos ya que permite controlar, diagnosticar y reparar equipos incluso si el sistema operativo no arranca. Es por esto que genera cierta discusión en cuanto al uso de este módulo, ya que si no es gestionado correctamente presenta un vector de riesgo crítico del sistema por el control tan profundo que tiene del mismo.

4) ¿Qué es coreboot?¿Qué productos lo incorporan?¿Cuáles son las ventajas de su utilización?

Coreboot es un proyecto de código abierto que busca reemplazar el sistema propietario UEFI. Está diseñado para ser ligero y ejecutar las tareas mínimas necesarias para cargar y correr un sistema operativo moderno y es respaldado por la Free Software Foundation.

Actualmente es utilizado en servidores de gran escala, supercomputadoras y sistemas IoT debido a su rapidez y bajo consumo de recursos. Además, desde 2013 se utiliza en la línea de laptops Chromebook.

Las principales ventajas de coreboot son

- Alta velocidad de arranque
- Seguridad, ya que tiene código auditable y no contiene sistemas propietarios ocultos.
- Flexibilidad, ya que permite utilizar distintos payloads.
- Transparencia y libertad. Al ser de código abierto los avances se comparten entre todos los usuarios y no se depende del firmware propietario.

Linker

1) ¿Qué es un linker? ¿Qué hace?

En la compilación de código, un linker es un programa que se encarga de combinar los distintos elementos intermedios que componen al programa final en un único ejecutable. Entre los diferentes componentes que se encarga de enlazar se encuentran las librerías y objetos, además de información de depuración como definiciones de símbolos, ubicación de código y datos de variables. El linker es el que permite conectar las definiciones de funciones y métodos que utilizará el programa con sus implementaciones en los archivos correspondientes.

2) ¿Qué es la dirección que aparece en el script del linker? ¿Por qué es necesaria?

En el script del linker se establece 0x7c00 como la dirección de memoria a partir de la cual se cargará el código para calcular referencias y posiciones absolutas correctamente. Se elige esa dirección específica ya que ahí es donde la BIOS carga los sectores de arranque.

3) Compare la salida de objdump con hd, verifique donde fue colocado el programa dentro de la imagen.

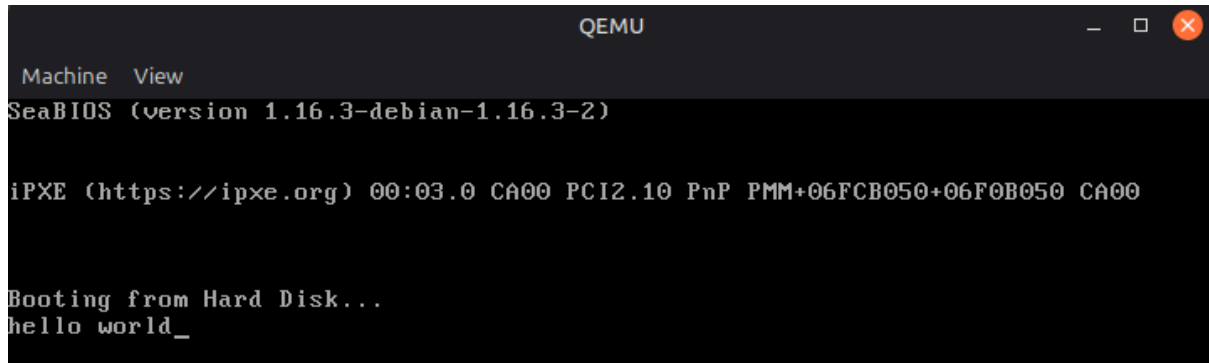


Figura 1.1 Ejecución de la imagen en QEMU

```

user@Device01: ~/programacion/Tiny_admins/tp3
user@Device01:~/programacion/Tiny_admins/tp3$ objdump -S main.o

main.o:          file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <loop-0x5>:
.code16
    mov $msg, %si
    0:  be 00 00 b4 0e          mov     $0xeb40000,%esi

0000000000000005 <loop>:
    mov $0x0e, %ah
loop:
    lodsb
    5:  ac                    lods    %ds:(%rsi),%al
    or %al, %al
    6:  08 c0                or      %al,%al
    jz halt
    8:  74 04                je      e <halt>
    int $0x10
    a:  cd 10                int     $0x10
    jmp loop
    c:  eb f7                jmp     5 <loop>

000000000000000e <halt>:
halt:
    hlt
    e:  f4                hlt

000000000000000f <msg>:
    f:  68 65 6c 6c 6f      push    $0x6f6c6c65
    14:  20 77 6f            and     %dh,0x6f(%rdi)
    17:  72 6c              jb      85 <msg+0x76>
    19:  64                fs
    ...
user@Device01:~/programacion/Tiny_admins/tp3$ █

```

Figura 1.2 objdump de main.o

```

user@Device01: ~/programacion/Tiny_admins/tp3
user@Device01:~/programacion/Tiny_admins/tp3$ hd main.img
00000000  be 0f 7c b4 0e ac 08 c0 74 04 cd 10 eb f7 f4 68 |..|.....t.....h|
00000010  65 6c 6c 6f 20 77 6f 72 6c 64 00 66 2e 0f 1f 84 |ello world.f....|
00000020  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 |....f.....f|
00000030  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000040  00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f |...f.....f..|
00000050  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
00000060  00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 |.f.....f....|
00000070  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 |....f.....f|
00000080  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000090  00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f |...f.....f..|
000000a0  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
000000b0  00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 |.f.....f....|
000000c0  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 |....f.....f|
000000d0  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
000000e0  00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f |...f.....f..|
000000f0  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
00000100  00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 |.f.....f....|
00000110  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 |....f.....f|
00000120  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000130  00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f |...f.....f..|
00000140  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
00000150  00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 |.f.....f....|
00000160  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 |....f.....f|
00000170  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000180  00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f |...f.....f..|
00000190  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
000001a0  00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 |.f.....f....|
000001b0  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 |....f.....f|
000001c0  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
000001d0  00 00 00 66 2e 0f 1f 84 00 00 00 00 00 66 2e 0f |...f.....f..|
000001e0  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
000001f0  00 66 2e 0f 1f 84 00 00 00 00 0f 1f 00 55 aa |.f.....U.|
00000200
user@Device01:~/programacion/Tiny_admins/tp3$ █

```

Figura 1.2 impresión hexadecimal de main.img

Se puede ver como el programa está ubicado al principio de la imagen, se ve el valor **f4** que equivale a la instrucción *halt*, la cadena “hello world” que se va a imprimir por pantalla, padding para completar los 512 bytes que debe ocupar un sector de arranque y **55 aa** al final para indicar a la BIOS que se trata de un sector booteable.

4) Grabar la imagen en un pendrive y probarla en una pc y subir una foto



Figura 1.3 booteo de main.img desde una unidad USB

5) ¿Para qué se utiliza la opción `--oformat binary` en el linker?

El argumento `--oformat binary` del linker especifica el formato del objeto que se creará. Al utilizar `binary` se genera el código directo en binario, de forma que la BIOS pueda leerlo correctamente, en lugar de generar un ejecutable ELF o Executable and Linkable Format que se utiliza en los programas normales.

Debugging

Utilizando `gdb` se pueden poner breakpoints antes y después de las interrupciones que escriben los valores en pantalla para ver cómo se construye el texto con cada ciclo del loop.


```
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050

Booting from Floppy...
_

Output/messages
user@Device01: ~/programacion/Tiny_admins/tp3

Breakpoint 1, 0x00007c00 in ?? ()
Assembly
0x00007c00 ? mov $0xeb47c0f,%esi
0x00007c05 ? lods %ds:(%esi),%al
0x00007c06 ? or %al,%al
0x00007c08 ? je 0x7c0e
0x00007c0a ? int $0x10
0x00007c0c ? jmp 0x7c05
0x00007c0e ? hlt
0x00007c0f ? push $0x6f6c6c65
0x00007c14 ? and %dh,0x6f(%edi)
0x00007c17 ? jb 0x7c05

Breakpoints
[1] break at 0x00007c00 for *0x7c00 hit 1 time
Expressions
History
Memory
Source
Stack
[0] from 0x00007c00
Threads
[1] id 1 from 0x00007c00
Variables
>>>
```

```
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050

Booting from Floppy...
_

Output/messages
user@Device01: ~/programacion/Tiny_admins/tp3

Breakpoint 2, 0x00007c0a in ?? ()
Assembly
0x00007c0a ? int $0x10
0x00007c0c ? jmp 0x7c05
0x00007c0e ? hlt
0x00007c0f ? push $0x6f6c6c65
0x00007c14 ? and %dh,0x6f(%edi)
0x00007c17 ? jb 0x7c05
0x00007c19 ? add %ah,%fs:0x2e(%esi)
0x00007c1d ? nopl 0x0(%eax,%eax,1)
0x00007c25 ? nopw %cs:0x0(%eax,%eax,1)
0x00007c2f ? nopw %cs:0x0(%eax,%eax,1)

Breakpoints
[1] break at 0x00007c00 for *0x7c00 hit 1 time
[2] break at 0x00007c0a for *0x7c0a hit 1 time
[3] break at 0x00007c0c for *0x7c0c hit 1 time
Expressions
History
Memory
Source
Stack
[0] from 0x00007c0a
Threads
[1] id 1 from 0x00007c0a
Variables
>>>
```

```
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050

Booting from Floppy...
h_

Output/messages
user@Device01: ~/programacion/Tiny_admins/tp3

Breakpoint 3, 0x00007c0c in ?? ()
Assembly
0x00007c0c ? jmp 0x7c05
0x00007c0e ? hlt
0x00007c0f ? push $0x6f6c6c65
0x00007c14 ? and %dh,0x6f(%edi)
0x00007c17 ? jb 0x7c05
0x00007c19 ? add %ah,%fs:0x2e(%esi)
0x00007c1d ? nopl 0x0(%eax,%eax,1)
0x00007c25 ? nopw %cs:0x0(%eax,%eax,1)
0x00007c2f ? nopw %cs:0x0(%eax,%eax,1)
0x00007c39 ? nopw %cs:0x0(%eax,%eax,1)

Breakpoints
[1] break at 0x00007c00 for *0x7c00 hit 1 time
[2] break at 0x00007c0a for *0x7c0a hit 1 time
[3] break at 0x00007c0c for *0x7c0c hit 1 time
Expressions
History
Memory
Source
Stack
[0] from 0x00007c0c
Threads
[1] id 1 from 0x00007c0c
Variables
>>>
```



```
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050
Booting from Floppy...
h_

user@Device01: ~/programacion/Tiny_admins/tp3
Output/messages
Breakpoint 2, 0x00007c0a in ?? ()
Assembly
0x00007c0a ? int $0x10
0x00007c0c ? jmp 0x7c05
0x00007c0e ? hlt
0x00007c0f ? push $0x6f6c6c65
0x00007c14 ? and %dh,0xf(%edi)
0x00007c17 ? jb 0x7c05
0x00007c19 ? add %ah,%fs:0x2e(%esi)
0x00007c1d ? nopl 0x0(%eax,%eax,1)
0x00007c25 ? nopw %cs:0x0(%eax,%eax,1)
0x00007c2f ? nopw %cs:0x0(%eax,%eax,1)
Breakpoints
[1] break at 0x00007c00 for *0x7c00 hit 1 time
[2] break at 0x00007c0a for *0x7c0a hit 2 times
[3] break at 0x00007c0c for *0x7c0c hit 1 time
Expressions
History
Memory
Source
Stack
[0] from 0x00007c0a
Threads
[1] id 1 from 0x00007c0a
Variables
>>>
```

```
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050
Booting from Floppy...
he_

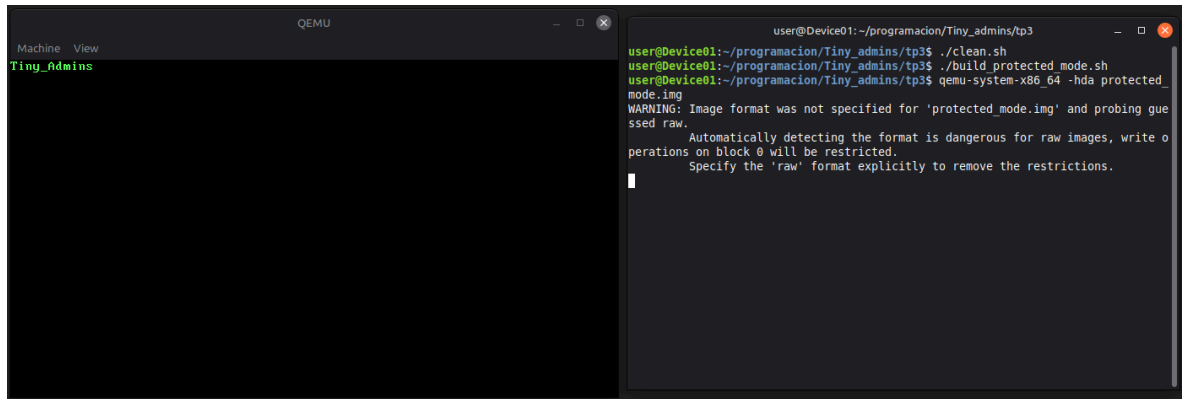
user@Device01: ~/programacion/Tiny_admins/tp3
Output/messages
Breakpoint 3, 0x00007c0c in ?? ()
Assembly
0x00007c0c ? jmp 0x7c05
0x00007c0e ? hlt
0x00007c0f ? push $0x6f6c6c65
0x00007c14 ? and %dh,0xf(%edi)
0x00007c17 ? jb 0x7c05
0x00007c19 ? add %ah,%fs:0x2e(%esi)
0x00007c1d ? nopl 0x0(%eax,%eax,1)
0x00007c25 ? nopw %cs:0x0(%eax,%eax,1)
0x00007c2f ? nopw %cs:0x0(%eax,%eax,1)
0x00007c39 ? nopw %cs:0x0(%eax,%eax,1)
Breakpoints
[1] break at 0x00007c00 for *0x7c00 hit 1 time
[2] break at 0x00007c0a for *0x7c0a hit 2 times
[3] break at 0x00007c0c for *0x7c0c hit 2 times
Expressions
History
Memory
Source
Stack
[0] from 0x00007c0c
Threads
[1] id 1 from 0x00007c0c
Variables
>>>
```

```
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050
Booting from Floppy...
hello world_

user@Device01: ~/programacion/Tiny_admins/tp3
Output/messages
Breakpoint 3, 0x00007c0c in ?? ()
Assembly
0x00007c0c ? jmp 0x7c05
0x00007c0e ? hlt
0x00007c0f ? push $0x6f6c6c65
0x00007c14 ? and %dh,0xf(%edi)
0x00007c17 ? jb 0x7c05
0x00007c19 ? add %ah,%fs:0x2e(%esi)
0x00007c1d ? nopl 0x0(%eax,%eax,1)
0x00007c25 ? nopw %cs:0x0(%eax,%eax,1)
0x00007c2f ? nopw %cs:0x0(%eax,%eax,1)
0x00007c39 ? nopw %cs:0x0(%eax,%eax,1)
Breakpoints
[1] break at 0x00007c00 for *0x7c00 hit 1 time
[2] break at 0x00007c0a for *0x7c0a hit 11 times
[3] break at 0x00007c0c for *0x7c0c hit 11 times
Expressions
History
Memory
Source
Stack
[0] from 0x00007c0c
Threads
[1] id 1 from 0x00007c0c
Variables
>>>
```

Modo Protegido

- 1) Crear un código assembler que pueda pasar a modo protegido (sin macros).



```
Machine View
Tiny_Admins

user@Device01:~/programacion/Tiny_admins/tp3$ ./clean.sh
user@Device01:~/programacion/Tiny_admins/tp3$ ./build_protected_mode.sh
user@Device01:~/programacion/Tiny_admins/tp3$ qemu-system-x86_64 -hda protected_mode.img
WARNING: Image format was not specified for 'protected_mode.img' and probing guesssed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
```

Al ejecutar la imagen generada con `protected_mode.s` se puede como se entra al modo protegido e imprime los valores cargados en el buffer de memoria VGA.

- 2) ¿Cómo sería un programa que tenga dos descriptores de memoria diferentes, uno para cada segmento (código y datos) en espacios de memoria diferenciados?

En modo real, código y datos comparten el mismo espacio de memoria porque las direcciones se calculan como:

$$\text{segmento} \times 16 + \text{offset}$$

En modo protegido, esto deja de existir. Cada descriptor de la GDT define:

- Una base real
- Un límite
- Permisos de acceso

Por lo tanto, dos segmentos pueden apuntar a regiones físicas totalmente distintas aunque usen el mismo offset.

Por ejemplo:

Segmento	Base	Límite	Uso
Código	0x00000000	64 KB	Instrucciones
Datos	0x00100000	64 KB	Variables

Descriptores:

`gdt_code: dq 0x00409A000000FFFF ; base 0x00000000`

`gdt_data: dq 0x004092001000FFFF ; base 0x00100000`

Entonces:

- `cs:0x0000` accede a la dirección física `0x00000000`
- `ds:0x0000` accede a la dirección física `0x00100000`

Esto demuestra que en modo protegido los segmentos ya no se solapan y cada uno referencia su propio espacio de memoria independiente.

3) **Cambiar los bits de acceso del segmento de datos para que sea de solo lectura, intentar escribir, ¿Qué sucede? ¿Qué debería suceder a continuación? Verificarlo con gdb.**

El byte de acceso del descriptor de datos normalmente es:

- `10010010b = 0x92` (lectura y escritura)

Si se cambia a:

- `10010000b = 0x90` (solo lectura)

El descriptor queda:

- `gdt_data: dq 0x00CF90001000FFFF`

Luego, en modo protegido:

```
mov ax, 0x10
mov ds, ax
mov byte [0], 0x55
```

Es decir, el procesador detecta que se intenta escribir en un segmento configurado como sólo lectura y genera una excepción:

- `#GP` — General Protection Fault

Como el programa no tiene configurada una IDT (tabla de interrupciones), ocurre la siguiente secuencia:

1. Se produce `#GP`.
2. El procesador intenta llamar al handler de esa excepción.
3. No encuentra IDT válida → genera `#DF` (Double Fault).
4. Tampoco existe handler para eso → ocurre un Triple Fault.
5. La CPU se reinicia automáticamente.

En QEMU se observa que la máquina se reinicia sola.

4) **En modo protegido, ¿Con qué valor se cargan los registros de**

segmento ? ¿Por qué?

En modo real, los registros de segmento contienen direcciones. En modo protegido, los registros de segmento contienen selectores.

Un selector es un valor que referencia una entrada dentro de la Global Descriptor Table (GDT). Ese valor no representa una dirección física ni lógica, sino un índice que el procesador utiliza para localizar el descriptor correspondiente al segmento que se quiere utilizar.

Cuando se carga un selector en un registro de segmento, el procesador:

1. Busca el descriptor indicado por ese selector dentro de la GDT.
2. Obtiene de ese descriptor la base del segmento, su límite y los permisos de acceso.
3. Almacena internamente esa información para utilizarla en la traducción de direcciones y en la verificación de protección.

Por este motivo, en modo protegido los registros de segmento se cargan con selectores y no con direcciones, ya que la dirección real y las condiciones de acceso al segmento quedan definidas por el descriptor asociado en la GDT.

Conclusión

En este trabajo se analizó el recorrido completo desde el arranque del sistema hasta la ejecución en modo protegido, comprendiendo el rol del firmware, el proceso de generación de una imagen booteable y el uso del linker para ubicar correctamente el código en memoria.

La parte central permitió observar de forma práctica cómo el procesador cambia su forma de direccionar memoria al pasar de modo real a modo protegido, utilizando descriptores en la Global Descriptor Table con base, límite y permisos propios para cada segmento. Asimismo, la implementación en hardware real (ThinkPad) destacó la importancia de una inicialización rigurosa de registros y la gestión de compatibilidad Legacy.

Finalmente, al forzar una escritura en un segmento configurado como sólo lectura, se verificó el funcionamiento del mecanismo de protección por hardware, evidenciando cómo el procesador detecta el acceso inválido y se reinicia ante la falta de manejo de excepciones. Esto permitió comprender en la práctica cómo se implementa la protección de memoria en arquitectura x86.

Link del repositorio utilizado: https://github.com/Igna124/Tiny_admins